



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang



Network Training (5.2.1)

Optimization:

A popular algorithm Adam (adaptive momentums) is combination of RMSProp (Root Mean Square Propagation) and Momentum. Momentum approach avoid getting stuck in critical points.

- **Momentum:** This method continues the movement from the previous iteration, adjusted by the negative gradient:

$$\begin{aligned} v^{(\tau+1)} &\leftarrow \lambda v^{(\tau)} - (1 - \lambda) \nabla E_n^{(\tau)} \\ w^{(\tau+1)} &\leftarrow w^{(\tau)} + \eta v^{(\tau+1)} \end{aligned}$$

decay factor

where λ is a hyperparameter, and $v^{(\tau+1)}$ is a moving average of $-\nabla E_n^{(\tau)}$. Momentum v is the accumulation of all previous gradients, with earlier gradients weighted less than more recent ones. Momentum helps accelerate convergence by maintaining the direction of the previous gradient. while the Lookahead method updates the gradient descent: $v^{(\tau+1)} = \lambda v^{(\tau)} - (1 - \lambda) \nabla E_n^{(\tau+1)}$ to accelerate convergence toward a minimum.

Using a smaller learning rate in steep directions prevents overshooting, while a larger learning rate in flat directions speeds up convergence.

Adaptive Gradient Algorithm

- **Adagrad:** The learning rate for each parameter is divided by the root mean square of its previous derivatives:

$$\begin{aligned} \mathbf{r}^{(\tau+1)} &\leftarrow \mathbf{r}^{(\tau)} + \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)} \\ \mathbf{w}^{(\tau+1)} &\leftarrow \mathbf{w}^{(\tau)} - \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \odot \nabla E_n^{(\tau)} \end{aligned}$$

Hadamard product (element-wise product)

$$(A \odot B)_{ij} = (A)_{ij} (B)_{ij}.$$

$$\mathbf{r}^{(1)} = \nabla E_n^{(0)} \odot \nabla E_n^{(0)}$$

$$\mathbf{r}^{(2)} = \mathbf{r}^{(1)} + \nabla E_n^{(1)} \odot \nabla E_n^{(1)}$$

$$\mathbf{r}^{(\tau+1)} = \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2$$

the root mean square of accumulated gradients

The update term can be expressed as:

$$\begin{aligned} \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} &= \frac{\eta}{\sqrt{\tau+1}} \odot \frac{1}{\sqrt{\frac{1}{\tau+1} \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2}} \\ &= \frac{\eta}{\sqrt{\sum_{i=0}^{\tau} \nabla E_n^{(i)} \odot \nabla E_n^{(i)}}} \end{aligned}$$

time delay by parameters

- Each parameter has its own learning rate. (element-wise product)
- Automatically adapts learning rates, especially useful for sparse data
- Step size depends on gradient magnitude, not update count.
- emphasizes the **relative change** of the gradient rather than its absolute value, which highlights variations in the gradient over time.

$$\frac{\eta}{\sqrt{\sum_{i=0}^{\tau} \nabla E_n^{(i)} \odot \nabla E_n^{(i)}}} \odot \nabla E_n^{(\tau)}$$

the square root of the accumulated average of past squared first derivatives. the current first derivative (the gradient)

Approximating the the ratio between the first derivative and the second derivative (Newton method update rule).

- First term: learning rate shrinks over time (time decay).
- Second term: adapts to past squared gradients – smaller slopes shrink faster.
- Step size depends on relative past updates, not just the current gradient.
- Risk: learning rate may shrink too early and slow down training.



Network Training (5.2.1)

- In **Adagrad**, the denominator grows continuously over time, which makes the learning rate shrink and may lead to premature stopping.

$$\mathbf{r}^{(\tau+1)} \leftarrow \mathbf{r}^{(\tau)} + \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)}$$
$$r^{(\tau+1)} = \sum_{i=0}^{\tau} (\nabla E_n^{(i)})^2$$

- Effective for **sparse features** and **high-dimensional data**.
- Valuable in NLP: Frequent words (the, is, and...) get smaller updates. Rare words (mitochondria, quasar...) retain larger learning rates.
- Learning rate keeps decreasing over time (monotonic decay).
- **RMSProp** replaces r with an **exponential moving average** so that it only considers the recent average of squared gradients.

$$r^{(\tau+1)} = \lambda r^{(\tau)} + (1 - \lambda)(\nabla E_n^{(\tau)})^2$$

Exponential decay

$$r^{(\tau+1)} = (1 - \lambda)(\nabla E_n^{(\tau)})^2 + \lambda(1 - \lambda)(\nabla E_n^{(\tau-1)})^2 + \dots$$
$$\mathbf{w}^{(\tau+1)} = \lambda \mathbf{w}^{(\tau)} - (1 - \lambda) \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \odot \nabla E_n^{(\tau)}$$

- **Adam – Adaptive Moment Estimation**

Then, The Adam optimizer combines the advantages of both RMSProp and Momentum by maintaining moving averages of the gradients and their second moments, which use exponential decay rates to apply bias corrections.

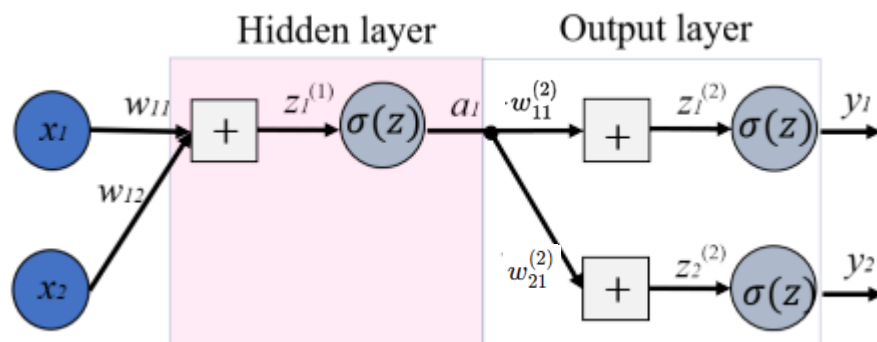
$$\mathbf{v}^{(\tau+1)} = \lambda_1 \mathbf{v}^{(\tau)} + (1 - \lambda_1) \nabla E_n^{(\tau)}$$
$$\mathbf{r}^{(\tau+1)} = \lambda_2 \mathbf{r}^{(\tau)} + (1 - \lambda_2) \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)}$$
$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \mathbf{v}^{(\tau+1)}$$

This allows for adaptive learning rates across different parameters, improving convergence. Adam adjusts the learning rate for each parameter based on the estimates of the first and second moments of the gradients, making it more resilient to noisy gradients and suitable for handling sparse data.



Network Training (5.3.1)

- Error backpropagation



$$w^{(\tau+1)} \leftarrow w^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

$$\frac{\partial E_n}{\partial w_{11}} = ?$$

Chain rule

$$\frac{\partial E_n}{\partial w_{11}} = \frac{\partial E_n}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}}$$

$$\begin{cases} z_1^{(1)} = w_{11}x_1 + w_{12}x_2 \\ \frac{\partial z_1^{(1)}}{\partial w_{11}} = x_1 \end{cases}$$

Chain rule

$$\frac{\partial E_n}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \sigma'(z_1^{(1)})$$

Chain rule

$$\frac{\partial E_n}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \frac{\partial E_n}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} w_{11}^{(2)} + \frac{\partial E_n}{\partial z_2^{(2)}} w_{21}^{(2)}$$

Next layer is output layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \frac{\partial E_n}{\partial y_1} \frac{\partial y_1}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \frac{\partial E_n}{\partial y_2} \frac{\partial y_2}{\partial z_2^{(2)}} \end{cases}$$

or hidden layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \frac{\partial E_n}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \frac{\partial E_n}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \end{cases}$$

$$y = g(x), \quad z = h(y)$$

$$\Delta x \rightarrow \Delta y \rightarrow \Delta z$$

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

$$x = g(s), \quad y = h(s), \quad z = k(x, y)$$

$$\Delta s \rightarrow \Delta x, \Delta y \rightarrow \Delta z$$

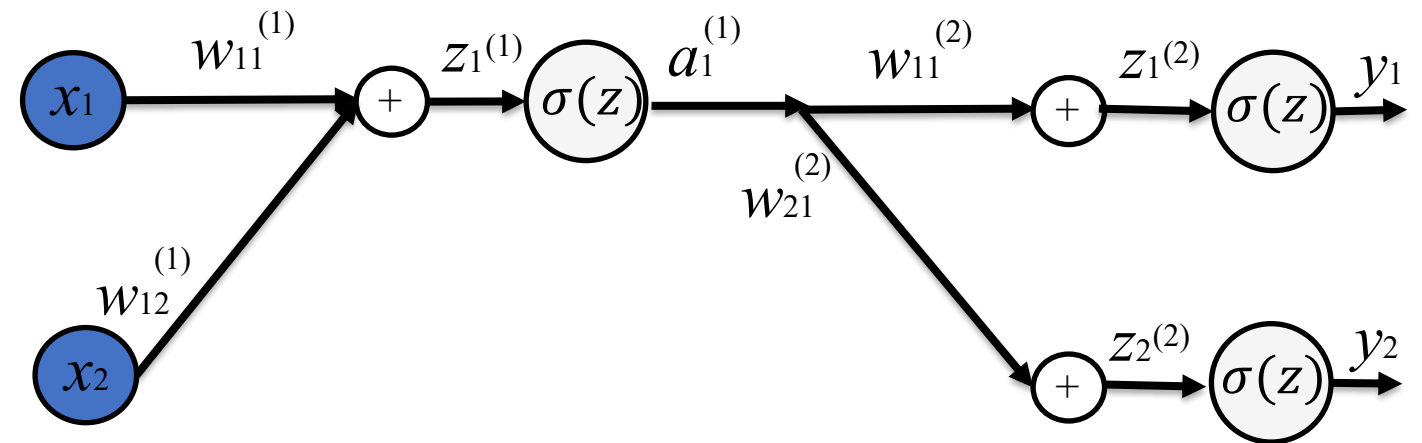
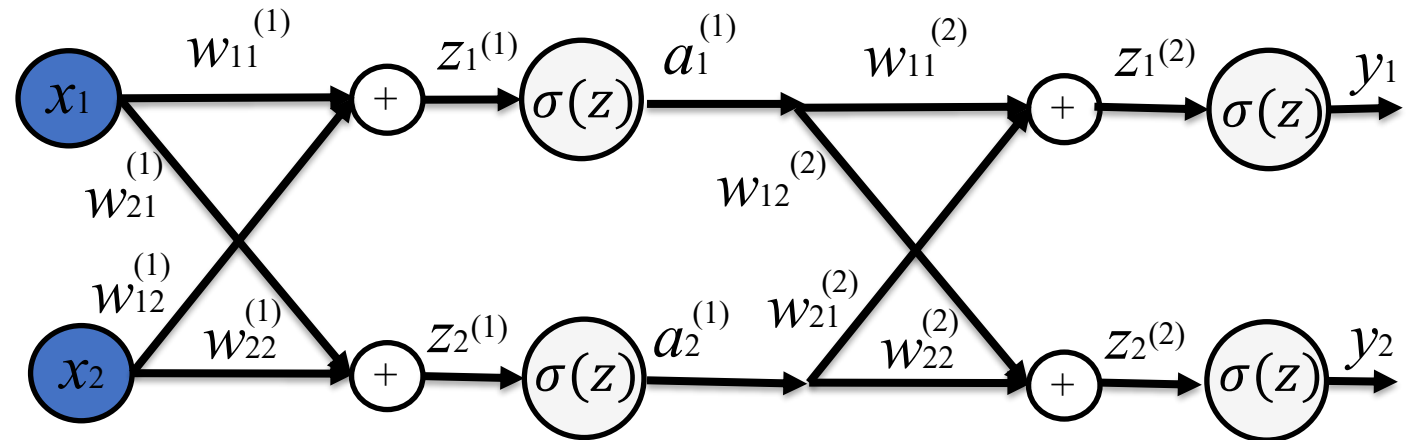
$$\frac{dz}{ds} = \frac{\partial z}{\partial x} \cdot \frac{dx}{ds} + \frac{\partial z}{\partial y} \cdot \frac{dy}{ds}$$



Network Training (5.3.1)

- Error backpropagation

$$\mathbf{x}_{2 \times 1} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{W}_{2 \times 2}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} \quad \mathbf{W}_{2 \times 2}^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \end{bmatrix}$$



$$\mathbf{x}_{2 \times 1} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad \mathbf{W}_{1 \times 2}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \end{bmatrix} \quad \mathbf{W}_{2 \times 1}^{(2)} = \begin{bmatrix} w_{11}^{(2)} \\ w_{21}^{(2)} \end{bmatrix}$$

$$z_{1 \times 1}^{(1)} = \mathbf{W}_{1 \times 2}^{(1)} \mathbf{x}_{2 \times 1} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2$$

$$a_{1 \times 1}^{(1)} = \sigma(z_{1 \times 1}^{(1)})$$

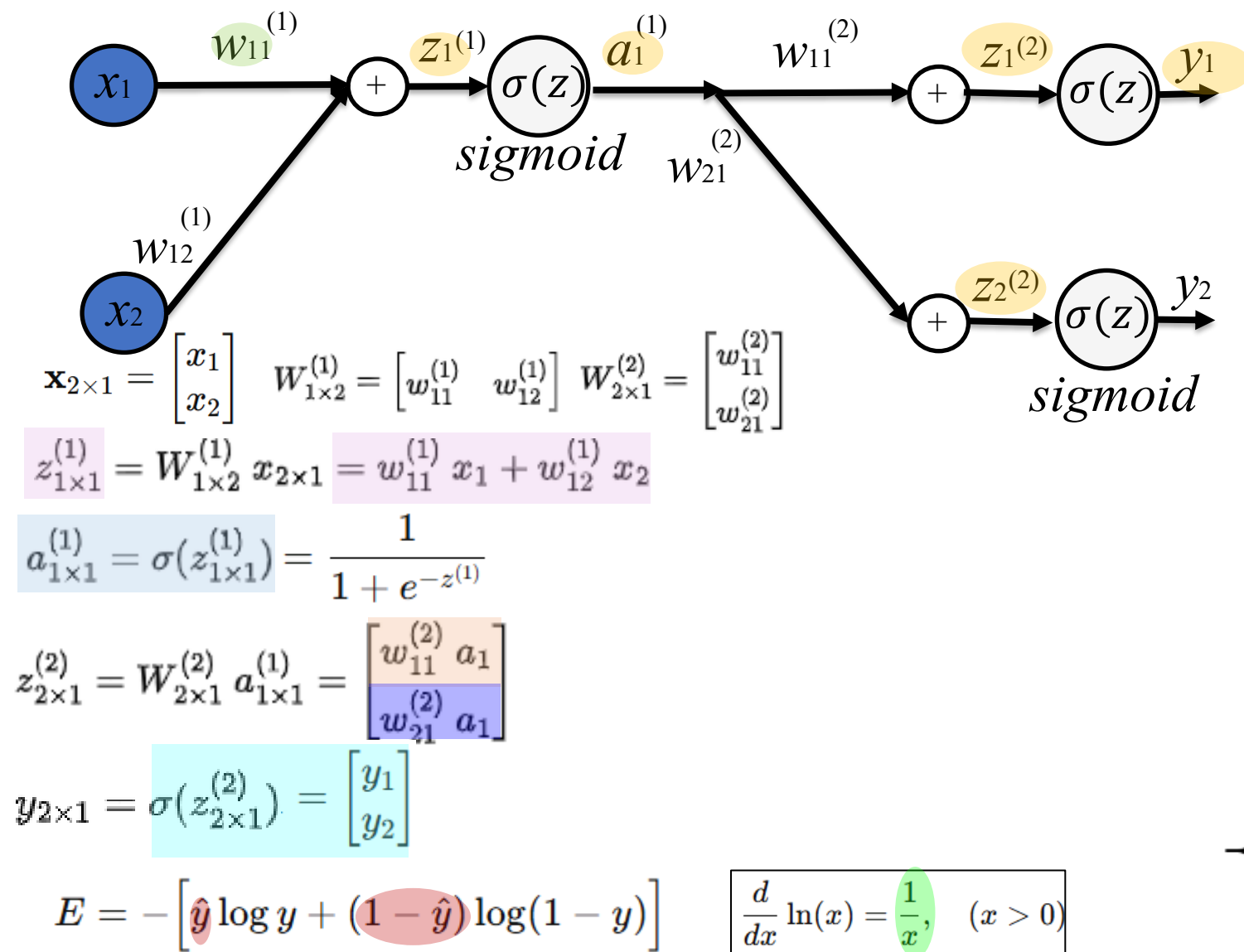
$$z_{2 \times 1}^{(2)} = \mathbf{W}_{2 \times 1}^{(2)} a_{1 \times 1}^{(1)} = w_{11}^{(2)} a_1 + w_{21}^{(2)} x_2$$

$$y_{2 \times 1} = \sigma(z_{2 \times 1}^{(2)})$$



Network Training (5.3.1)

- Error backpropagation



$$w^{(\tau+1)} \leftarrow w^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

$$E_n = E_n(\mathbf{W}_{1 \times 2}^{(1)}, \mathbf{W}_{2 \times 1}^{(2)}) = E_n(w_{11}^{(1)}, w_{12}^{(1)}, w_{11}^{(2)}, w_{21}^{(2)})$$

$$\frac{\partial E_n}{\partial w_{11}^{(1)}}, \quad \frac{\partial E_n}{\partial w_{12}^{(1)}}, \quad \frac{\partial E_n}{\partial w_{11}^{(2)}}, \quad \frac{\partial E_n}{\partial w_{21}^{(2)}}$$

$$\frac{\partial E_n}{\partial w_{11}} = ? \quad \text{Gotcha!}$$

Chain rule

$$\frac{\partial E_n}{\partial w_{11}} = \frac{\partial E_n}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}} = \frac{\partial E_n}{\partial z_1^{(1)}} \frac{\partial (w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2)}{\partial w_{11}} = \frac{\partial E_n}{\partial z_1^{(1)}} x_1$$

Chain rule

$$\frac{\partial E_n}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\partial E_n}{\partial a_1^{(1)}} \sigma'(z_1^{(1)})$$

Chain rule

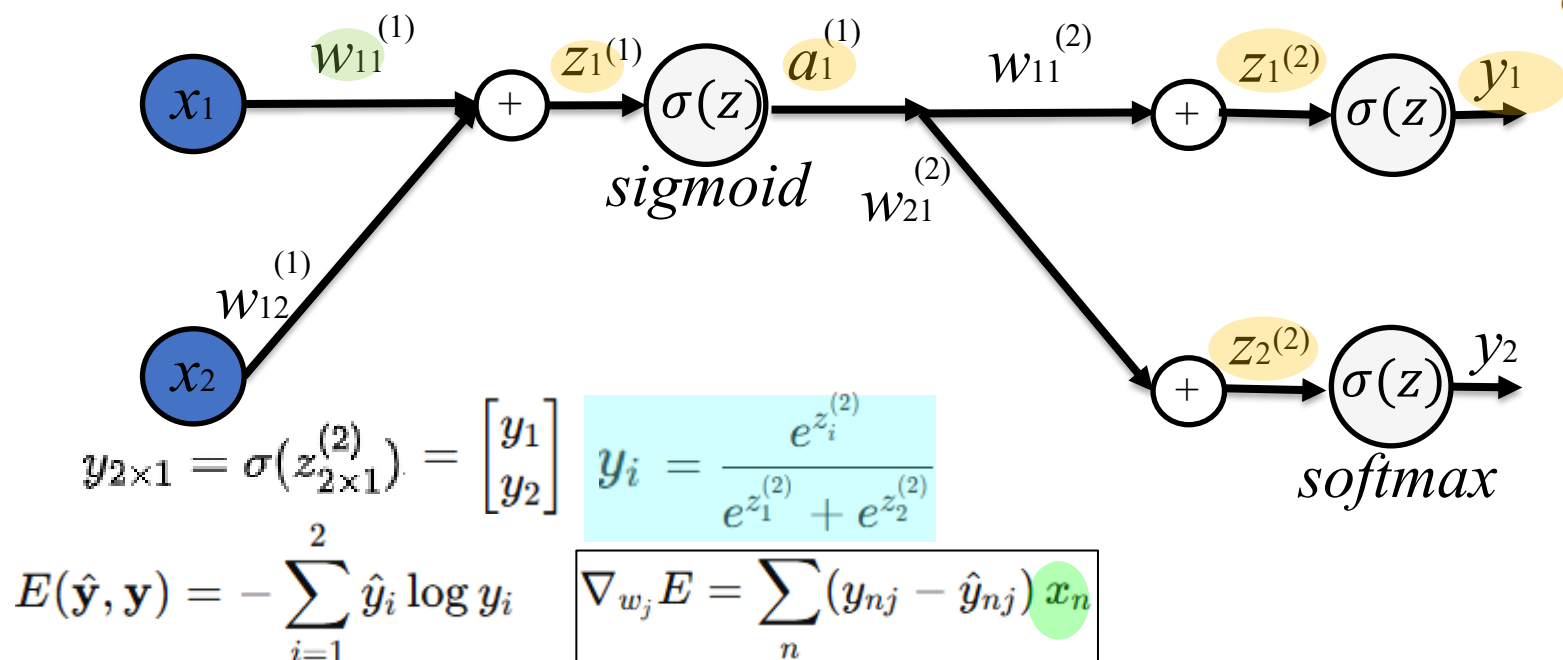
$$\frac{\partial E_n}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} + \frac{\partial E_n}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} = \frac{\partial E_n}{\partial z_1^{(2)}} w_{11}^{(2)} + \frac{\partial E_n}{\partial z_2^{(2)}} w_{21}^{(2)}$$

Next layer is output layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \frac{\partial E_n}{\partial y_1} \frac{\partial y_1}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \frac{\partial E_n}{\partial y_2} \frac{\partial y_2}{\partial z_2^{(2)}} \end{cases} \begin{cases} \frac{\partial E}{\partial y_1} = -\left(\frac{\hat{y}_1}{y_1} - \frac{1 - \hat{y}_1}{1 - y_1}\right) = \frac{y_1 - \hat{y}_1}{y_1(1 - y_1)} \\ \frac{\partial y_1}{\partial z_1^{(2)}} = \sigma'(z_1^{(2)}) = y_1(1 - y_1) \end{cases} \rightarrow \frac{\partial E}{\partial \mathbf{z}^{(2)}} = \mathbf{y} - \hat{\mathbf{y}}$$

Network Training (5.3.1)

- Error backpropagation



Next layer is output layer

$$\begin{cases} \frac{\partial E_n}{\partial z_1^{(2)}} = \sum_{i=1}^2 \frac{\partial E_n}{\partial y_i} \frac{\partial y_i}{\partial z_1^{(2)}} \\ \frac{\partial E_n}{\partial z_2^{(2)}} = \sum_{i=1}^2 \frac{\partial E_n}{\partial y_i} \frac{\partial y_i}{\partial z_2^{(2)}} \end{cases}$$

$$\frac{\partial E_n}{\partial y_i} = -\frac{\hat{y}_i}{y_i} \quad \frac{\partial y_i}{\partial z_j} = y_i (\delta_{ij} - y_j)$$

Quotient Rule (除法公式)

$$h(x) = \frac{u(x)}{v(x)}$$

$$h'(x) = \frac{u'v - uv'}{v^2}$$

$$y_i = \frac{u}{v}, \quad u = e^{z_i}, \quad v = \sum_k e^{z_k} = S$$

$$\frac{\partial y_i}{\partial z_j} = \frac{u'v - uv'}{v^2}$$

$$u' \rightarrow \frac{\partial u}{\partial z_j} = \frac{\partial e^{z_i}}{\partial z_j} = e^{z_i} \cdot \delta_{ij}$$

$$v' \rightarrow \frac{\partial v}{\partial z_j} = \frac{\partial}{\partial z_j} \sum_k e^{z_k} = e^{z_j}$$

$$\frac{\partial y_i}{\partial z_j} = \frac{e^{z_i} \delta_{ij} S - e^{z_i} e^{z_j}}{S^2} = \frac{e^{z_i}}{S} (\delta_{ij} - \frac{e^{z_j}}{S}) = y_i (\delta_{ij} - y_j)$$

$u_i = \frac{e^{z_i}}{S} \downarrow$

$$\frac{\partial E}{\partial z_j} = \sum_{i=1}^K \left(-\frac{\hat{y}_i}{y_i} \right) \cdot y_i (\delta_{ij} - y_j) = -\sum_{i=1}^K \hat{y}_i (\delta_{ij} - y_j)$$

$$\delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

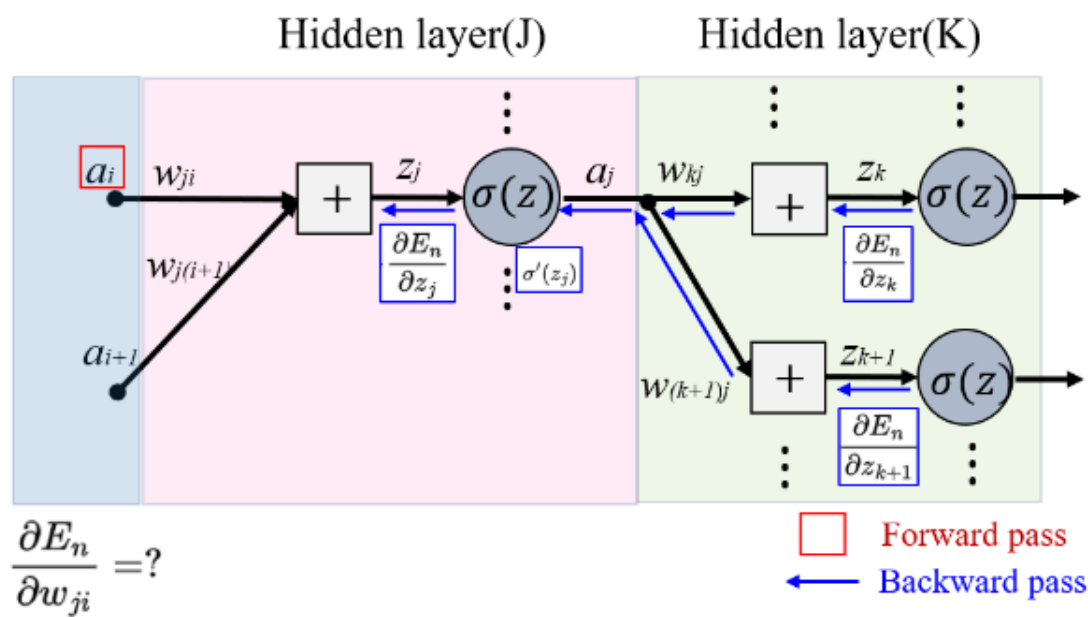
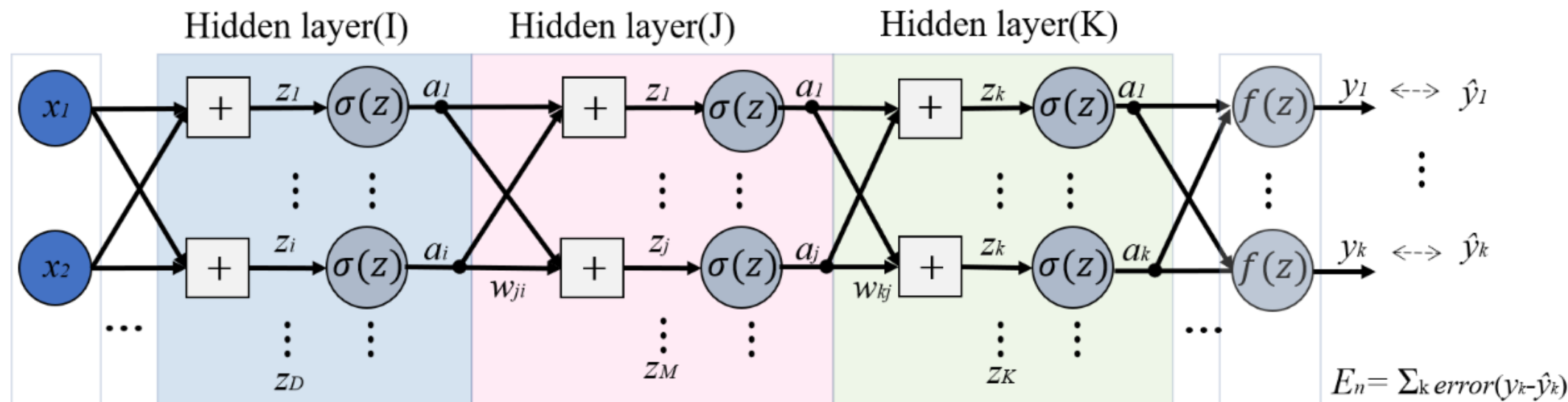
$$\frac{\partial E}{\partial z_j} = -\hat{y}_j + y_j \sum_{i=1, i \neq j}^K \hat{y}_i \quad \sum_{i=1}^K \hat{y}_i = 1$$

$$\frac{\partial E}{\partial z_j} = y_j - \hat{y}_j \rightarrow \frac{\partial E}{\partial \mathbf{z}^{(2)}} = \mathbf{y} - \hat{\mathbf{y}}$$



Network Training (5.3.1)

- Error backpropagation



Chain rule

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial z_j} \frac{\partial z_j}{\partial w_{ji}}$$

$$\begin{cases} z_j = \dots + w_{ji}a_i + w_{j(i+1)}a_{i+1} + \dots \\ \frac{\partial z_j}{\partial w_{ji}} = a_i \end{cases}$$

Chain rule

$$\frac{\partial E_n}{\partial z_j} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial z_j} = \frac{\partial E_n}{\partial a_j} \sigma'(z_j)$$

Chain rule

$$\begin{aligned} \frac{\partial E_n}{\partial a_j} &= \dots + \frac{\partial E_n}{\partial z_k} \frac{\partial z_k}{\partial a_j} + \frac{\partial E_n}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial a_j} + \dots = \sum_k \frac{\partial E_n}{\partial z_k} \frac{\partial z_k}{\partial a_j} \\ &= \sum_k \frac{\partial E_n}{\partial z_k} w_{kj} \end{aligned}$$

Next layer (K) is output layer or hidden layer

$$\frac{\partial E_n}{\partial z_k} = \frac{\partial E_n}{\partial y_k} \frac{\partial y_k}{\partial z_k} \quad \text{or} \quad \frac{\partial E_n}{\partial z_k} = \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial z_k}$$

$k = 1, 2, \dots, K$

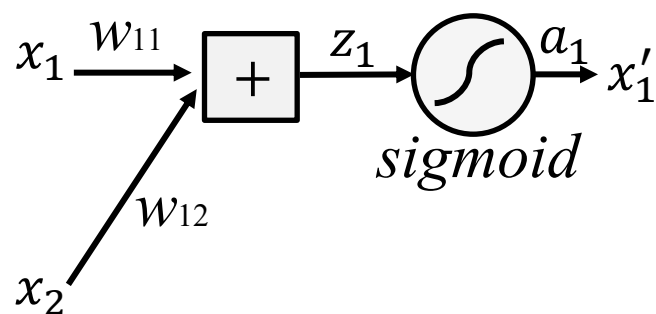
$$\rightarrow \frac{\partial E_n}{\partial w_{ji}} = a_i \sigma'(z_j) \sum_k w_{kj} \frac{\partial E_n}{\partial z_k}$$

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}$$



Example of Error Backpropagation

- Binary classification task



Input: $x = [1, 2]^T$

Initial weights: $W = [0.2, -0.5]$

No bias term

Activation: **sigmoid**

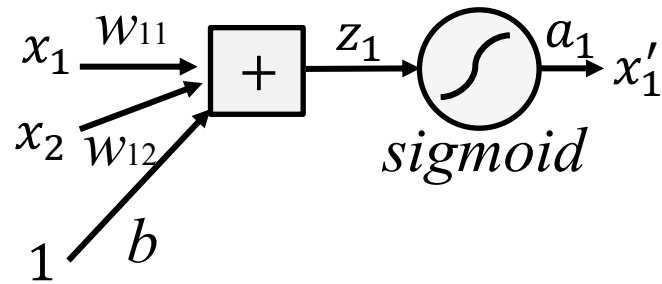
Target label: $\hat{y} = 1$

Learning rate: $\eta = 0.1$



Example of Error Backpropagation

- Binary classification task



Input: $x = [1, 2]^T$

Initial weights: $W = [0.2, -0.5]$

Initial weight: $b = 0$

Activation: **sigmoid**

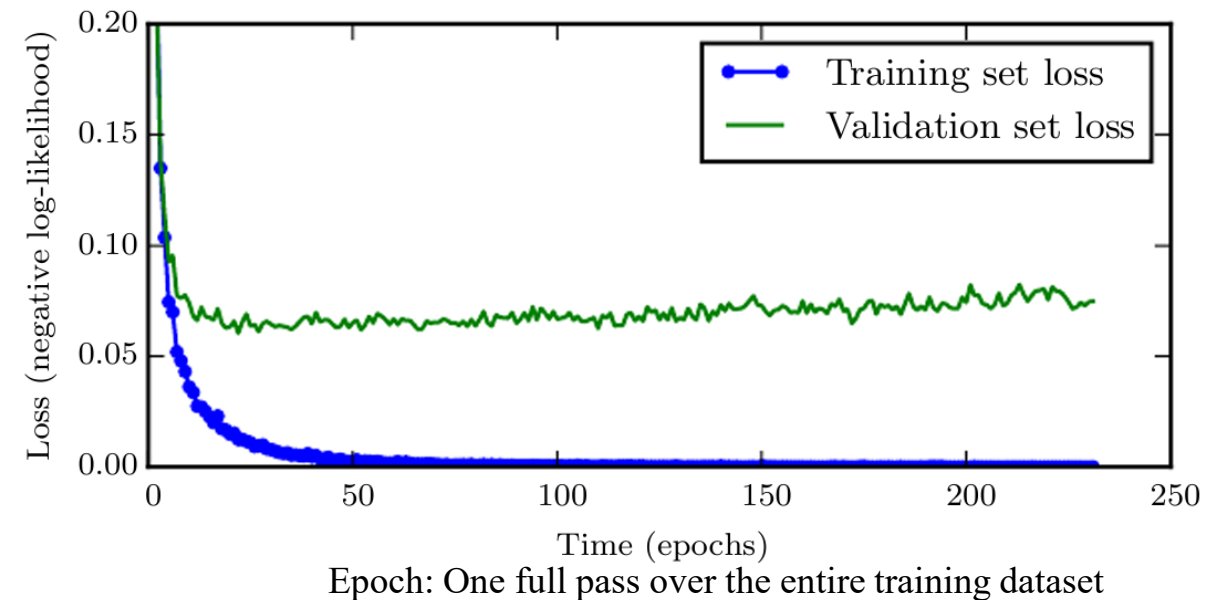
Target label: $\hat{y} = 1$

Learning rate: $\eta = 0.1$



Network Training (5.3.1)

- Learning curve: how a model's performance evolves during training
- Accuracy Curve: reflects **predictive performance**
- **Good training performance does not** guarantee good performance on **unseen data**.
- We need a **validation set** to evaluate how well the model **generalizes** (概括) to **unseen data** during training.
- A model should perform well **beyond** the training data — not just memorize it.



- Loss values may **oscillate** during training, especially under **mini-batch gradient descent**.
- Causes:
 1. Each mini-batch has a **different sample distribution**, leading to noisy gradient estimates.
 2. A **large learning rate** can cause the model to overshoot and oscillate near minima.
 3. Some data points are **harder to fit**, temporarily increasing loss.
- The model should **converge on average**, even if the curve is not perfectly smooth.



Network Training (5.3.1)

- **Underfitting:**
Performs poorly on both training and validation data
Indicates insufficient learning or low model capacity
- **Overfitting:**
Performs **well** on training data but **poorly** on unseen data (poor generalization)
The model **fits the training data too closely**, including noise
- Early Stopping:
Stop when validation loss reaches its **minimum**

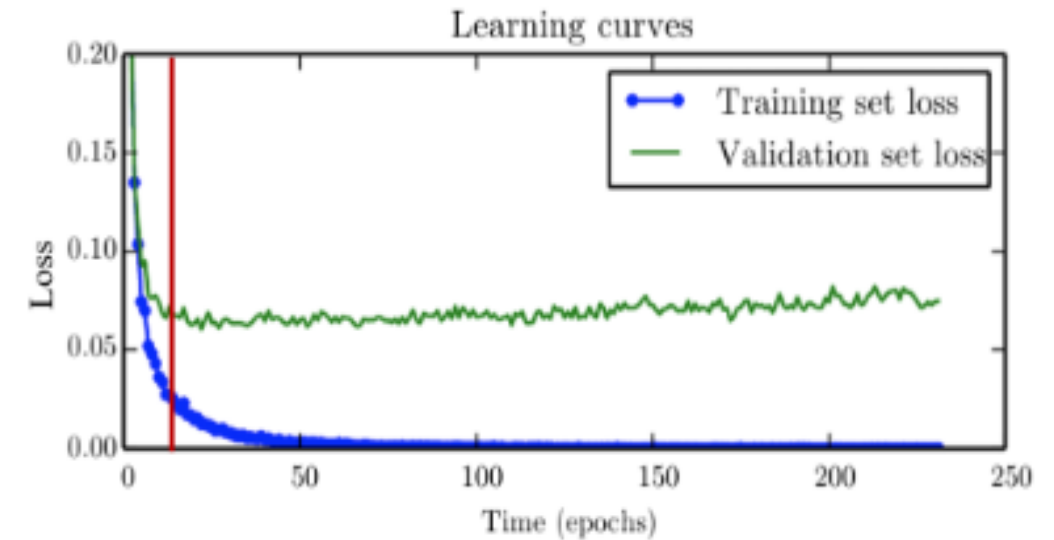


Fig. 13: Although the training objective consistently decreases over time, the average loss on the validation set may eventually start to increase again. Therefore, training can be halted at the point of the smallest error with respect to the validation dataset (red line) to achieve a network that exhibits good generalization performance. This approach is known as early stopping.

Network Training (5.3.1)

- **L2 Penalty**

1. Imposes a cost on large weight magnitudes, thereby encouraging solutions with **smaller norms** that remain **closer to the origin**.
2. Reflects a **preference for simpler models** with smoother decision boundaries.

$$E_w(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

- **Data Fitting Term (ML)**

1. **Stochastic Gradient Descent (SGD)** minimizes the data-fitting error $E_D(\mathbf{w})$ alone
2. The solution $\mathbf{w}^*$ corresponds to the point where the **training error** is minimized.

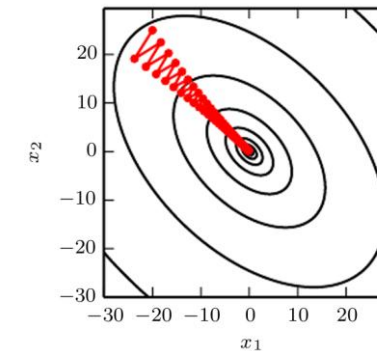
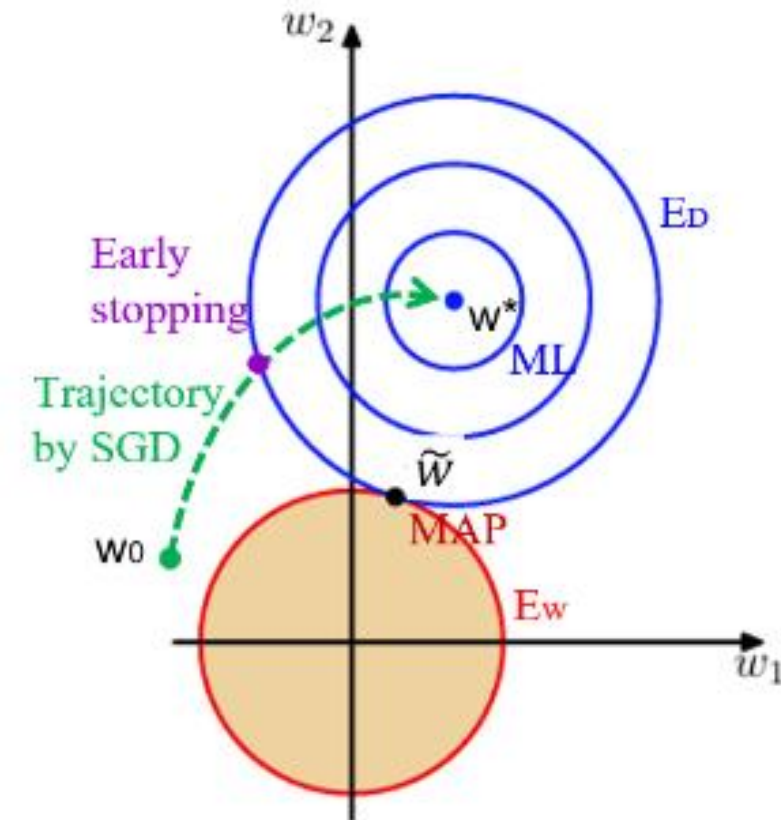
$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (y_n - \mathbf{w}^T x_n)^2$$

- **Regularized Objective (MAP)**

Represents a **trade-off** between **data fitting** and **model complexity**.

$$E(\mathbf{w}) = E_D(\mathbf{w}) + \lambda E_w(\mathbf{w})$$

- **Early stopping** serves as a form of regularization by constraining the training duration, and **approximates L2 regularization** when optimization starts near the origin and progresses outward in a **stable, convex** loss landscape.



- The quadratic cost surface exhibits different curvature across dimensions
- Gradient descent follows the steepest direction, causing zigzagging and slow convergence along elongated contours
- Normalization rescales input features to achieve smoother descent paths



Model Capacity

- **Model capacity** refers to the model's ability to represent a wide range of functions (Complexity of model)
- Underfitting
Model too simple to capture patterns
Both **training error** and **generalization error** are **high**
- Overfitting
Model fits training data extremely well (**low training error**)
Generalization error increases due to fitting noise
→ Require larger dataset and proper regularization

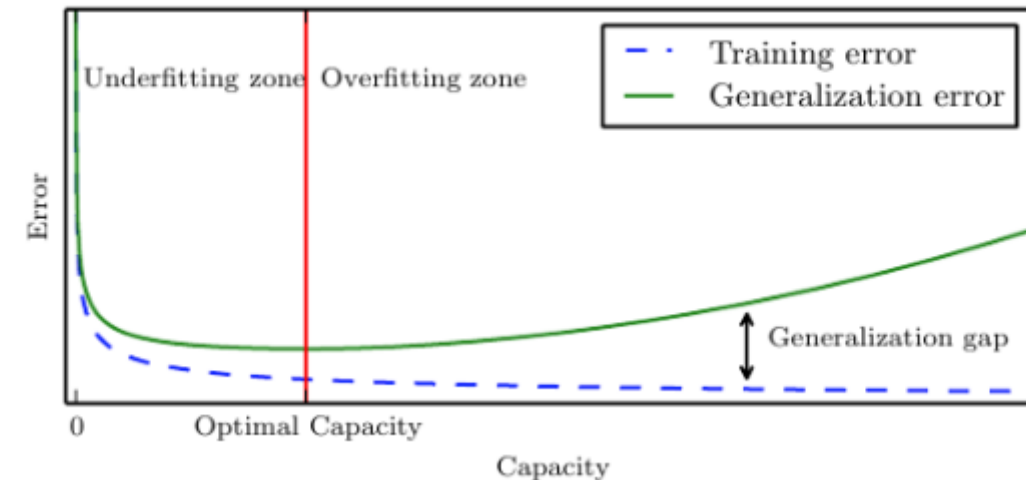


Fig. 12: Capacity (Complexity of model) refers to a model's ability to fit a wide range of functions. At the left end both training and generalization errors are high, indicating *underfitting*. As capacity increases, training error decreases, but the gap between training and generalization errors grows. When this gap becomes too large, we enter the *overfitting* regime, where the model's capacity exceeds the optimal level, harming generalization



Network Training (5.3.1)

Algorithm 1 Model Training, Validation, and Testing

- 1: Initialize model parameters θ
 - 2: Set hyperparameters: learning rate η , number of epochs E , batch size b
 - 3: Split dataset into training set D_{train} , validation set D_{val} , and test set D_{test}
 - 4: **for** epoch = 1 to E **do**
 - 5: Shuffle D_{train}
 - 6: **for** each mini-batch $B \subset D_{train}$ **do**
 - 7: Forward pass: Compute predictions $\hat{y} = f(X_B; \theta)$
 - 8: Compute loss $\mathcal{L}(y_B, \hat{y})$
 - 9: Backward pass: Compute gradients $\nabla_{\theta} \mathcal{L}$
 - 10: Update parameters: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$
 - 11: **end for**
 - 12: Calculate training loss $L_{train}(epoch)$
 - 13: Calculate validation loss $L_{val}(epoch)$
 - 14: Update learning curves
 - 15: **end for**
 - 16: Evaluate model on test set D_{test}
 - 17: Compute test metrics (e.g., accuracy, F1 score)
-